



GEBZE TECHNICAL UNIVERSITY

COMPUTER ENGINEERING

CSE484 Introduction to Natural Language Processing

Fall 2024-2025

Homework #1 Report

**Statistical Language Models Using N-Grams for
Syllables and Characters**

Name Surname	Selin Duru DERİNDAG
Student Number	200104004801

I. Introduction

In this assignment, we are to create and evaluate a statistical language model for the Turkish language using N-grams of syllables and characters.

Language modelling with N-grams is a statistical approach where a model learns the probability of a token or sequence of tokens by analyzing the frequency of occurrence in a corpus. An N-gram model of order N estimates the likelihood of a sequence of a sequence of words or characters based on the previous $N-1$ elements, creating a probabilistic structure that reflects language patterns.

This assignment focuses on two distinct N-gram models: a syllable model and a character model. A syllable-based model divides words into the phonetic units based on the given language's rules. A character model treats each character as a singular unit.

In this assignment we will be creating the model from top to bottom, starting from preprocessing the data, calculating the n-grams, evaluating the perplexity of test sentences, and generating random sentences with our n-grams and perplexity tables.

The objective of this assignment is to understand and evaluate the effectiveness of statistical language modeling techniques. While doing this assignment we are able gain a practical understanding of the theoretical knowledge we gained in class such as perplexity calculation and smoothing techniques. Lastly, we evaluate the pros and cons of different types of language models and n-grams to see first-hand what is effective and what is not.

II. Design and Implementation

a. Program Flow

The program starts by preprocessing the test data by taking the percentage of the entire data you want to use; in case you would like to work with a subset of the data. At this point, the data is also syllabized and characterized. This processed data is written to a file.

The syllable model is created first. The syllabized data is read from the file and fed into the syllable model object. Upon object creation, the unigrams, bigrams, and trigrams are created and smoothed. Afterwards the perplexity of the smoothed n-grams is calculated, and the random sentences are generated. The perplexities and randomly generated sentences are written to a file.

The same process repeats for the character model.

b. Splitting the data

I split the data according to the number of lines in the corpus. The data is split according to the total percentage of the corpora that will be used (this percentage can be changed within the code). Among the subset (or entire set of the corpora that is being used, 95% of it is used for training and 5% of it reserved for testing.

```
def _split_train_test(corpora_usage_percentage, test_percentage=5):
    print("Splitting train and test data...")
    total_lines = int(TOTAL_NUMBER_OF_LINES * (corpora_usage_percentage /
100))
    test_lines_count = int(total_lines * test_percentage / 100)
    train_lines_count = total_lines - test_lines_count

    with open(CORPORA_FILE, "r", encoding="utf-8") as file:
        with open(TRAINING_SET, "w", encoding="utf-8") as train_file:
            with open(TESTING_SET, "w", encoding="utf-8") as test_file:
                for line_number, line in enumerate(file, 1):
                    if line_number <= train_lines_count:
                        train_file.write(line)
                    else:
                        test_file.write(line)
                        if line_number == total_lines:
                            break

    return train_lines_count, test_lines_count
```

Figure: Function to split data into testing and training subsets.

Its inputs are the total percentage of corpora usage, and the percentage that will be used for testing (which is set to default 5%). The split data is separated into two files and returns the line counts of the training and testing files. The data is split according to the total number of lines of the corpus (or subset of the corpus) to avoid any sentences or words being cut off in the middle. I also utilized the `html2text` library to remove any HTML tag within the dataset. While characterizing and syllabizing the words, I added three special tokens to indicate the end of a word `</w>`, the start of a sentence `<s>` and the end of a sentence `</s>`.

```
def _characterize_text(text):
    words = WORD_SPLIT_PATTERN.findall(text)
    characterized_text = ''

    for word in words:
        if word.isalpha():
            characters = characterize(word)
            characterized_text += characters + EOW_V2
        elif word in EOS_PUNCTUATIONS:
            last_occurrence = characterized_text.rfind(EOW_V2)
            if last_occurrence != -1:
                characterized_text = characterized_text[:last_occurrence] + ' ' + EOS + ' '

    return characterized_textanager.switchView('timesheet')>Timesheet</div>
</label>
```

Figure: function utilizing the characterize function

I used the syllabize and characterize functions in the process by slitting the data text into words and calling the characterize/syllabize function for it. I made sure to handle any punctuation and the ends of words and sentences. This function takes the data as an input and returns the characterized/syllabized text fully processed and ready to write to the file.

c. Syllabizing the words

I wrote a function to split the words according to the Turkish language syllable rules which I defined as regular expressions. This function takes the words from the corpus one by one and returns the syllabized word.

```
def syllabilize(word):
    if len(word) < 3:
        return word

    for rule in rules:
        if re.match(rule["pattern"], word, re.IGNORECASE):
            if len(word) > rule["length"]:
                return f"{word[:rule['length']]} {syllabilize(word[rule['length']:])}"
            else:
                return word

    return word
```

Figure: snippet of syllabize function

d. Characterizing the words

I wrote a similar function to the syllabizing function that takes words one by one as input and returns the characters as a string separated by whitespace.



```
def characterize(word):  
    word = word.lower()  
  
    if len(word) < 2:  
        return word  
  
    return " ".join(list(word))
```

Figure: Characterize function.

e. Language Model Class

The language model class does all the calculations and basically all the inner work of the program. This class (model) takes the preprocessed data as (as tokens) an input and gives the perplexities and generates random sentences as an output. This class includes functions such as `_init_ngram` to create the n-gram tables as well as the smoothed n-gram tables, `calculate_perplexity`, and `generate_random_sentences`.

f. `_init_ngram` Function

This function creates the n-gram tables, smooths them, and keeps track of the zero-frequency probabilities.

```
def _init_ngram(self, n):  
    print(f"Calculating {n}-grams...\n")  
    ngrams = list(nltk.ngrams(self.tokens, n))  
    ngram_table = Counter(ngrams)  
  
    print(f"Smoothing {n}-grams...\n")  
    smoothed_ngram_table, zero_counts = smoothing(ngram_table)  
  
    return ngrams, smoothed_ngram_table, zero_counts
```

Figure: n-gram initialization function

This function takes the gram-number as an input and creates the corresponding table as a dictionary. I calculated the n-grams using Python's nltk (natural language toolkit) and stored the count of all n-grams to create the table using Python's built in collection function, Counter.

g. Smoothing n-grams

```
def smoothing(table):  
    frequencies_of_counts = _count_counts(table)  
  
    smoothed_table = {}  
  
    for gram, ngram_count in table.items():  
        N_c = frequencies_of_counts.get(ngram_count, 1)  
        N_c_plus_1 = frequencies_of_counts.get(ngram_count + 1, 1)  
        smoothed_count = (((ngram_count + 1) * N_c_plus_1) / N_c)  
        smoothed_table[gram] = smoothed_count  
  
    N_1 = frequencies_of_counts.get(1, 1)  
    P_zero_frequency = N_1 / sum(table.values())  
    return smoothed_table, P_zero_frequency
```

Figure: Smoothing function

To smooth the n-grams I implemented the Good-Turing intuition formula. This function only takes the n-gram table as an input.

‘gram’ is the token,

‘ngram_count’ is the count of the token,

‘N_c’ is the frequency of a specific count in the n-gram table,

I applied the formula in my function by using the get() function, which returns the counts that I want, or returns 1 if it does not exist. The reason I returned 1 by default is to avoid any divisions by zero. At the end of the function, I calculate the probability of the zero-frequency tokens (which will be used in calculating probability later on).

h. Calculating probability

To calculate the probability of unigrams, I simply divided the count of a unigram (or the zero-count probability, if the n-gram does not exist) by the total number of unigrams.

Calculating the bi- and tri-grams were a little more complicated.

```
def _trigram_probability(prev1, prev2, curr, smoothed_trigram_tables,
    smoothed_bigram_tables, bigram_zero_counts):
    numerator = smoothed_trigram_tables.get((prev1, prev2, curr), bigram_zero_counts)
    denominator = smoothed_bigram_tables.get((prev1, prev2), bigram_zero_counts)
    return float(numerator) / float(denominator)
```

Figure: Tri-gram probability function.

This function takes the two previous gram tokens, as well as the current one. It also takes the smoothed bigram and trigram tables and the zero-probability frequency of bigrams.

In the numerator I have the count of the entire trigram or the zero count of the two previous grams and in the denominator, I have the bigram of the two previous grams or the zero-probability frequency, if it does not exist.

The bigram function is like this, except I use only one previous gram and unigram zero counts.

i. Perplexity Calculation

To calculate the perplexities of the n-grams, I first tried to apply the exp-log-sum perplexity formula given to us in class one-to-one, however, this resulted in a division-by-zero error. To remedy this, I applied the following formula instead:

$$\exp \left(\frac{-1}{N} \sum_{i=1}^N \log P(\text{syllable}_i) \right)$$

```
def trigram_perplexity(tokens, smoothed_trigram_tables, smoothed_bigram_tables, bigram_zero_counts):
    number_of_tokens = len(tokens)
    prob_sum_logs = 0
    prev1, prev2 = None, None
    for curr in tokens:
        if prev1 is not None and prev2 is not None:
            syllable_probability = _trigram_probability(prev1, prev2, curr, smoothed_trigram_tables,
smoothed_bigram_tables, bigram_zero_counts)
            prob_sum_logs += -math.log(syllable_probability)
            prev1, prev2 = prev2, curr
    return math.exp(prob_sum_logs / number_of_tokens)
```

Figure: Trigram Perplexity Function

This function takes the smoothed bigram and trigram functions, the bigram zero counts, and the test sentence in token form as an input. It implements the above formula (which is for unigrams) by using the probability of the two previous tokens as a bigram and continues to the next trigram one token at a time.

The bigram perplexity function is like this, using only a previous unigram and unigram zero counts.

```
def calculate_perplexity(self, test_cases):
    with open(self.OUTPUT_FOLDER+"/perplexity", "w", encoding='utf-8') as f:
        i = 1
        for sentence in test_cases:
            test_tokens = sentence.split()
            if len(test_tokens) > 2:
                f.write("Test Sentence {}: {}\n".format(i, postprocess(sentence)))
                f.write("Unigram perplexity: {}\n".format(unigram_perplexity(test_tokens,
self.smoothed_unigram_tables, self.unigrams, self.unigram_zero_counts)))
                f.write("Bigram perplexity: {}\n".format(bigram_perplexity(test_tokens,
self.smoothed_bigram_tables, self.smoothed_unigram_tables, self.unigram_zero_counts)))
                f.write("Trigram perplexity: {}\n".format(trigram_perplexity(test_tokens,
self.smoothed_trigram_tables, self.smoothed_bigram_tables, self.bigram_zero_counts)))
                i += 1
    print("Perplexity calculation complete. See results in the appropriate model's output folder.")
```

Figure: Language Model function to utilize the perplexity calculation functions.

These functions return the perplexity of a certain test sentence, and the language model writes the output to a file.

j. Random Sentence Generation

To generate random sentences, I set a limit of 10 tokens for each sentence for testing, and to generate these tokens, I sorted the top 5 most probable grams and randomly chose one of them.

```
def generate_random_unigram(unigram_table, max_length):  
    sentence = []  
  
    for _ in range(max_length):  
        top_unigrams = [ngram for ngram in sorted(unigram_table.items(), key=lambda x: x[1], reverse=True)[:5]  
                        if ngram[0][0] != END_OF_SENTENCE and ngram[0][0] != END_OF_WORD]  
        if not top_unigrams:  
            break  
        next_word = random.choice(top_unigrams)[0][0]  
        sentence.append(next_word)  
  
    return ' '.join(sentence)
```

Figure: Function to generate random sentences from unigrams.

Using unigrams to generate random sentences, I ended up using the top 5 unigrams repeatedly because the next step has no connection to the previous token at any point.

```
def generate_random_trigram(trigram_tables, max_length):  
    sentence = []  
  
    start_words = random.choice([key for key in trigram_tables.keys() if key[0] == END_OF_SENTENCE and key[1]  
!= END_OF_SENTENCE])  
    sentence.extend(start_words)  
  
    for _ in range(max_length - 2):  
        prev_words = tuple(sentence[-2:])  
  
        next_word_choices = [  
            (key[2], trigram_tables[key])  
            for key in trigram_tables.keys()  
            if key[:2] == prev_words and key[2] != END_OF_SENTENCE  
        ]  
  
        if not next_word_choices:  
            break  
  
        top_5_trigrams = sorted(next_word_choices, key=lambda x: x[1], reverse=True)[:5]  
        words, weights = zip(*top_5_trigrams)  
  
        next_word = random.choices(words, weights=weights, k=1)[0]  
  
        sentence.append(next_word)  
  
    sentence.append(END_OF_SENTENCE)  
    return ' '.join(sentence)
```

Figure: Generating random sentences from trigrams

For bigrams and trigrams, I was able to get the top 5 next most probable grams in relation to the previous token(s).

```
def generate_random_sentences(self, no_of_sentences=10):  
    with open(self.OUTPUT_FOLDER+"/random_sentences", "w", encoding='utf-8') as file:  
        for sentence_no in range(1, no_of_sentences+1):  
            file.write(f"#{sentence_no}:\n")  
            file.write(f"Unigram: {generate_random_unigram(self.smoothed_unigram_tables,  
no_of_sentences)}\n")  
            file.write(f"Bigram: {generate_random_bigram(self.smoothed_bigram_tables,  
no_of_sentences)}\n")  
            file.write(f"Trigram: {generate_random_trigram(self.smoothed_trigram_tables,  
no_of_sentences)}\n\n")  
  
    print("Random sentences have been generated. See results in the appropriate model's output folder.")
```

Figure: Language Model's sentence generation function

The generation functions output the randomly generated sentence, which is plugged into the language model's function, which writes the output to a file.

III. Results and Tables

a. Perplexity Results

Syllable Model:

Test Sentences	Unigram Perplexity	Bigram Perplexity	Trigram Perplexity
kafkas kolordu komutanlığından iii	1765037524.8490627	16.975740696556343	12.36677392574727
kadar türk ü kurtarıp erhacı ya topladılar	1765037524.8490565	58.56791185935493	25.84609145233073
melekli de muhasara altında kalan	1765037524.8490627	20.999018677052398	7.058316730598997
ağustos tarihinde	1765037524.8490689	7.80879648540445	1.4082139022600813
ile yılları arasında gerçekleştirilen büyük göç süresince	1765037524.8490627	10.929003765153828	6.823633704581325

Character Model:

Test Sentences	Unigram Perplexity	Bigram Perplexity	Trigram Perplexity
kafkas kolordu komutanlığından iii	187145594209.47418	14.922481905120156	10.739636010040451
kadar türk ü kurtarıp erhacı ya topladılar	187145594209.47418	12.20384516262944	9.304771491140174
melekli de muhasara altında kalan	187145594209.47418	8.81390150057349	6.5169736724478255
ağustos tarihinde	187145594209.47485	10.644574111398597	7.479519655941166
ile yılları arasında gerçekleştirilen büyük göç süresince	187145594209.47485	9.472082832683546	5.262326902850524

b. Randomly Generated Sentences

Syllable Model:

Unigram	Bigram	Trigram
de de la le de la le le de le	or ta </w> a lan dı </s> i le </w>	sa nal </w> a na </w> gö re </w> de
de la le la le le le la la le	bir </w> i </w> o lu </w> i le </w>	bağ </w> ve </w> i çin </w> bir lik te
de le le la de la de la de de	i le </w> a la rı </w> ve </w> a	e mi ri mi ne </w> gö re </w> de
la de de le le le le le de la	i ki ler </s> bu lu nan </w> a lan	de ter jan </w> as ke ri </w> i çin
la le de la la le le de de la	bu lun du ğu </w> a ra sın da ğı	a yet te </w> o la rak </w> ta ra

Character Model:

Unigram	Bigram	Trigram
i n a i e n n i n a	k i </w> d i r ı n </w> k	ö y l e n </w> k a r ı
n a n a a a e n a n	y ı n i </w> i n i r a	ü ç ü m ü z i r </w> v
e i i a e e a a e n	b u n d e </w> a r a r	y o n u n u </w> i l e
i i n n n a i a n n	b a </w> d e l ı n d a	r a s y o n u </w> k a
e e e a e a e e i n	y a r ı r </w> a r ı r	n ü m ü z e n </w> b u

IV. Analysis and Conclusion

a. Perplexity Analysis

a. Character Model

- i. Unigram: The unigram perplexities are consistently too high, suggesting significant uncertainty in predicting individual characters without any context, such as previous tokens. This indicates that the unigram is unable to capture even the most basic character-level structure in Turkish.
- ii. Bigram: The bigram perplexity values can be considered acceptable as they range on average from 12~30 which shows that the model is able to capture common character transitions better than isolated characters. These results indicate that the bigram model can predict character pairs fairly.
- iii. Trigram: The trigram model has the lowest perplexity values ranging on average from 5~12, which can be considered ideal. This indicates that the trigram model can manage some degree of the Turkish language's morphological structure but cannot produce any coherent sentences.

b. Syllable Model

- i. Unigram: The syllable model's unigram perplexity, though the character model's, is still too high, suggesting that predicting isolated tokens without any context remains challenging in any model.

- ii. Bigram: The bigram perplexity for the syllable model ranges from around 10.9 to 47.3. These values are generally higher than those for the character bigrams, indicating that syllable-to-syllable transitions are more complex for the model to capture.
- iii. Trigram: The trigram perplexity (ranging from ~5.63 to ~15.07) is lower than the bigram's but still higher than that of the character trigrams. The syllable trigrams show improved predictability but struggle with sentence-level coherence, rarely producing anything with meaning, due to the high variability in syllable patterns.

b. Random Sentence Generation Analysis

a. Character Model

- i. Unigram: The character unigrams reflect random selections of common Turkish vowels and consonants. They are essentially meaningless as they lack any context, producing repetitive sequences that don't resemble words or coherent structures. The unigrams provide limited information about the language structure due to the lack of inter-character relationships.
- ii. Bigrams: The bigrams exhibit some semblance of word formation, as seen in segments like "bu" and "dir." However, these combinations often fail to form complete, meaningful words or phrases.
- iii. Trigrams: While the trigrams are somewhat coherent at the character level, they still lack overall linguistic meaning and often generate sequences that do not form meaningful or complete sentences.

b. Syllable Model

- i. Unigrams: The syllable unigrams produce repetitive syllables commonly found in Turkish, but like the character unigrams, they lack context. They highlight frequent syllables but do not convey any meaningful phrases or sentences, rendering them essentially nonsensical in isolation.
- ii. Bigrams: The syllable bigrams are more realistic than character bigrams in terms of forming partial word constructs but still lack grammatical context for coherent sentences.
- iii. Trigrams: These trigrams occasionally form meaningful phrases, albeit with some inconsistencies. They generally approximate Turkish linguistic patterns better than character trigrams, due to the inherent meaning carried by each syllable in Turkish.

c. General Comparison between models

a. Trade-offs between Syllables vs Characters for N-grams

- i. Linguistic Structure: Character models capture language structure at a finer level, focusing on individual character transitions. This approach is beneficial for languages with simpler morphological structures but poses challenges for Turkish, an agglutinative language with complex inflectional morphology. Syllable models capture more linguistic information per unit, aligning better with Turkish morphology. Since Turkish has a syllable-based structure with clear syllabic boundaries, syllable-based N-grams can better capture meaningful units like roots, suffixes, and entire words, which are critical in forming coherent sentences in Turkish.

- ii. Model Complexity and Vocabulary Size: Character models have a smaller vocabulary, allowing for simpler models but often limiting their ability to capture the nuances of Turkish syntax and semantics. Syllable Models have a more extensive vocabulary, enabling them to represent Turkish language nuances more accurately but potentially introducing sparsity and greater variability in training data.

d. General Comparison Between N-grams

- a. Unigrams: Provided redundant and meaningless sentences for both character and syllable models.
- b. Bigrams: Shows some improvement, forming partial words and recognizable character/syllable pairings, without being able to form complete sentences.
- c. Trigrams: Character model achieves almost coherent words while syllable model achieves coherent sentences sometimes.

e. Conclusion:

Best Model: The **syllable model** aligns well with Turkish's agglutinative nature, capturing meaningful linguistic units and producing more coherent sentence structures.

Best N-Gram Level: **Syllable trigrams** are the most suitable for Turkish, as they generate sentence-like structures with realistic transitions. They balance coherence and grammatical structure, providing plausible Turkish phrases and sentence fragments.

V. Citations and LLM Usage

Usage Details	Chat Link
Asking GPT what data structure is best to use for storing the n-gram tables.	https://chatgpt.com/share/6720cb71-f0f8-801-a060-cbd1cda75801
Wrote the syllabizing and characterizing functions with the help of GPT.*	(Could not find the link- probably accidentally deleted it)
Asked GPT to help in implementing the Good-Turing Smoothing function.*	https://chatgpt.com/share/67214ca9-e440-8001-bf16-5f8ac654abac
GPT wrote me a skeleton code for the Language Model class to get me started in the implementation.	https://chatgpt.com/share/67212dfb-1f28-8001-8fde-c162da62f090
GPT helped me implement the storage of the n-gram tables a dictionary.	
Asked GPT to help me in fixing the divide-by-zero error in the perplexity function.	https://chatgpt.com/share/672491cc-be60-8001-8181-d3c32bcbe6b6
Had GPT refactor some ugly code I wrote.	https://chatgpt.com/share/67249e58-9c3c-8001-b7a5-ac3d0b6f8fe8
Asked GPT to help me understand the random sentence generation instructions and create the functions.	https://chatgpt.com/share/6724d6ea-d874-8001-8de1-0022369a37e7
Report Writing Assistance (Introduction)	https://chatgpt.com/share/67250b54-a074-8001-9e16-00edfc796ae3
Report Writing Assistance (Conclusion)	https://chatgpt.com/share/672514a6-6864-8001-8fe1-6ca34758dcfe

* The functions written by GPT were not copied and pasted one-to-one as chat's answers were not sufficient 😊, I did have to fix and add many things, however chat did help e in getting the main idea or solving any problems I encountered while programming on my own.